INTERNSHIP PROJECT REPORT

ON

STUDENT MANAGEMENT SYSTEM

USING JAVA

Submitted in partial fulfillment of the requirements of the
Internship Project Submission

Submitted By	Saurav Singh
Internship Role	Java Developer Intern
Organization	Codec Technologies
Internship Duration	July 2026
Project Title	Student Management System using Java

2026

CERTIFICATE

This is to certify that the project entitled “Student Management System using Java” is prepared as an internship project by Saurav Singh in connection with the Java Developer Internship at Codec Technologies. The project demonstrates the application of core Java programming concepts, object-oriented programming, collections, input validation, exception handling, and file-based data persistence.

DECLARATION

I, Saurav Singh, declare that the project titled “Student Management System using Java” is an original academic internship project prepared for the purpose of demonstrating practical knowledge and skills in Java programming. The project documentation has been prepared based on the implemented project and the internship context.

Name: Saurav Singh

Role: Java Developer Intern

Date: _____

Signature: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Codec Technologies for providing me with the opportunity to complete an internship as a Java Developer Intern. This project helped me understand how core Java concepts can be applied to build a practical management application.

I am thankful to the project mentors and internship coordinators for providing guidance and a platform to improve my programming and problem-solving skills. I also thank my teachers, friends, and family members for their support and encouragement during the completion of this project.

ABSTRACT

The Student Management System is a console-based Java application developed to manage basic student records efficiently. The system provides features to add, view, search, update, and delete student information. Each student record contains a unique student ID, name, age, course, and email address.

The application is designed using object-oriented programming principles. Student data is managed through Java collections and persisted using file handling so that records can be retained between program executions. Input validation and basic error handling are included to improve usability and prevent invalid entries.

The project demonstrates practical implementation of classes and objects, encapsulation, constructors, methods, ArrayList, loops, conditional statements, switch-case logic, serialization, and exception handling. The application is suitable as a beginner-level Java project while still covering the fundamental concepts required for a Student Management System.

TABLE OF CONTENTS

1. INTRODUCTION

Student information is commonly maintained in educational institutions and training environments. Managing records manually can become inconvenient when the number of records increases. A simple software application can organize student information and make common operations such as searching, updating, and deleting records easier.

The Student Management System developed in this project is a menu-driven console application written in Java. It allows a user to perform basic CRUD operations: Create, Read, Update, and Delete. The program uses a Student class to represent student data and a StudentManager class to manage the collection of records.

The project also saves records using Java object serialization. This allows the application to retain saved student data in a local file and load it again when the program is restarted.

2. OBJECTIVES

- To develop a practical application using core Java.
- To understand and implement object-oriented programming concepts.
- To perform CRUD operations on student records.
- To use the Java Collections Framework for managing multiple records.
- To implement input validation for better usability.
- To use file handling for persistent storage of student data.
- To understand the structure of a small real-world Java application.

3. PROBLEM STATEMENT

Manual handling of student records can be repetitive and difficult to manage. A user may need to add new records, search for a specific student, modify information, or remove outdated records. The objective of this project is to create a simple Java-based system that performs these operations through a structured menu-driven interface.

4. SCOPE OF THE PROJECT

The current project focuses on basic student record management in a local console environment. It is designed primarily for learning and demonstration purposes. The application supports local data storage and can be extended in the future with a graphical user interface, database integration, authentication, and advanced reporting features.

5. TECHNOLOGIES USED

Technology	Usage	Purpose
Java	Programming Language	Core application development
OOP	Programming Paradigm	Organizing data and behavior
ArrayList	Collections Framework	Managing student records
Serialization	File Handling	Persistent local storage
Scanner	Input Utility	Accepting user input

Exception Handling	Error Management	Handling invalid data and I/O issues

6. SYSTEM REQUIREMENTS

Hardware Requirements:

- Computer/Laptop
- Minimum 4 GB RAM recommended
- Basic storage space for JDK and project files

Software Requirements:

- Java Development Kit (JDK) 17 or later
- VS Code, IntelliJ IDEA, Eclipse, or another Java IDE
- Windows, Linux, or macOS

7. METHODOLOGY

The project was developed using a modular approach. First, the required student attributes and operations were identified. A Student class was created to represent individual records. A StudentManager class was then designed to handle CRUD operations and file storage. Finally, a Main class was implemented to provide a menu-driven interface and collect user input.

- Requirement identification
- Class and data model design
- Implementation of CRUD operations
- Input validation
- File-based persistence
- Testing of individual features
- Documentation and project packaging

8. SYSTEM DESIGN AND ARCHITECTURE

The application follows a simple layered structure:

- Main.java: Handles the user interface, menu, input, and program flow.
- Student.java: Represents the student entity and encapsulates student attributes.
- StudentManager.java: Contains business logic for adding, viewing, searching, updating, deleting, loading, and saving student records.

Flow: User Input → Main Class → StudentManager → Student Records / File Storage

9. MODULES AND FEATURES

Add Student: Creates a new student record after validating the input. Duplicate student IDs are not allowed.

View All Students: Displays all available student records.

Search Student: Finds a student record using the unique student ID.

Update Student: Modifies the name, age, course, and email of an existing student.

Delete Student: Removes a student record from the collection.

Data Persistence: Stores student records in a local students.dat file using serialization.

Input Validation: Checks numeric input, positive values, non-empty text, and a basic email format.

10. OBJECT-ORIENTED PROGRAMMING CONCEPTS USED

Classes and Objects: The Student class acts as a blueprint for creating student objects.

Encapsulation: Student fields are declared private and accessed through getter and setter methods.

Constructors: The Student constructor initializes a new student object with required values.

Abstraction of Responsibilities: StudentManager keeps record-management logic separate from the user interface.

Methods: Separate methods are used for add, search, update, delete, save, and load operations.

11. DATA PERSISTENCE AND FILE HANDLING

The project uses Java object serialization to store student records locally. When the program starts, StudentManager attempts to load previously saved records from students.dat. When a record is added, updated, or deleted, the latest list is written back to the file.

This approach provides simple persistence without requiring a database server. It is appropriate for a small educational project and can later be replaced by JDBC-based database storage.

12. PROJECT STRUCTURE

```
Student-Management-System/
├── src/
│   ├── Main.java
│   ├── Student.java
│   └── StudentManager.java
├── README.md
├── .gitignore
└── students.dat (created automatically after saving records)
```

13. SOURCE CODE EXPLANATION

Student.java defines the data model. It contains fields for ID, name, age, course, and email, along with constructors, getter methods, setter methods, and a formatted toString() method.

StudentManager.java manages the ArrayList of Student objects. It provides methods for adding records, viewing records, searching by ID, updating details, deleting records, and saving/loading data through serialization.

Main.java contains the main method and menu logic. It repeatedly displays available options, reads the user choice, and calls the appropriate StudentManager method. Helper methods are used to validate numeric and text input.

14. TESTING AND VALIDATION

Test Case	Expected Result	Status
Add a new student	Student should be added successfully	Passed
Add duplicate ID	Duplicate record should be rejected	Passed
View records	All saved records should be displayed	Passed
Search existing ID	Correct student details should appear	Passed
Search invalid ID	Student not found message should appear	Passed
Update existing record	Student details should be updated	Passed
Delete existing record	Student should be removed	Passed
Restart after saving	Previously saved data should load	Passed
Invalid numeric input	Program should request valid input	Passed

15. RESULTS AND OUTPUT

The application successfully performs the planned CRUD operations through a menu-driven console interface. Student records can be created, viewed, searched, updated, and deleted. The program also retains saved data through file-based persistence.

For the final printed report, console screenshots of the following operations can be inserted in this section:

- Main menu screen
- Adding a student
- Viewing all students
- Searching for a student
- Updating a student
- Deleting a student

16. CONCLUSION

The Student Management System was successfully designed as a Java-based console application for managing student information. The project demonstrates the practical use of core Java and object-oriented programming concepts. It includes CRUD functionality, input validation, duplicate ID checking, and file-based data persistence.

Through this project, important programming concepts such as classes, objects, encapsulation, constructors, collections, methods, loops, conditional statements, exception handling, and serialization were applied in a practical application.

17. FUTURE SCOPE

- Develop a graphical user interface using JavaFX or Swing.
- Replace local file storage with MySQL using JDBC.
- Add login and authentication features.
- Generate student reports and statistics.
- Add sorting and filtering features.
- Develop a web-based version of the system.
- Add role-based access for administrators and staff.

18. REFERENCES

- Java Development Kit documentation and standard Java libraries.
- Project source code developed for the Student Management System.
- Internship offer letter and internship certificate provided for the Java Developer Internship at Codec Technologies.